02.6 Clasificación - Ejercicio para entregar sobre variedades vínicolas

Print to PDF

```
import matplotlib.pyplot as plt
%matplotlib inline
from IPython import display
import numpy as np
```

Se va a utilizar un archivo con información de variedades vinícolas

```
# Se preparan las columnas a tratar
import pandas as pd
df_wine = pd.read_csv('data/wine.csv', header=None)
df_wine.columns = ['Class label', 'Alcohol',
    'Malic acid', 'Ash',
    'Alcalinity of ash',
    'Magnesium', 'Total phenols',
    'Flavanoids', 'Nonflavanoid phenols',
    'Proanthocyanins',
    'Color intensity', 'Hue',
    'OD280/OD315 of diluted wines'
    ,'Proline']
# Se borra la clase 1
df_wine = df_wine[df_wine['Class label'] != 1]
df_wine.head()
# Se toma el campo 'Class label' como variable objetivo y
# Se toman los campos 'Alcohol' y 'OD280/OD315 of diluted wines' como
características sobre una matriz X
y = df_wine['Class label'].values
X = df_wine[['Alcohol', 'OD280/OD315 of diluted wines']].values
np.unique(y)
```

```
array([2, 3])
```

Presentar una vista del dataframe cargado

Preproceso de datos

NOTA A sklearn.metrics:__ Algunas métricas se definen esencialmente para tareas de clasificación binaria (por ejemplo, f1_score, roc_auc_score). En estos casos, por defecto solo se evalúa la etiqueta positiva, asumiendo por defecto que la clase positiva está etiquetada como 1 (aunque esto puede configurarse a través del **parámetro pos_label**).

Por lo tanto hay dos alternativas:

- **Se recodifica la variable objetivo y**, que contiene la etiqueta 'Class label' para que adopte los valores (0,1)
- **Este ejemplo se convierte en una clasificación binaria**, y se pueden mostrar sus métricas con sklearn.metrics

La otra posibilidad es:

- **No alterar los valores de y**
- Indicar en las llamadas a **sklearn.metrics** en **pos_label** la etiqueta que funciona como verdadero (3 por ejemplo).

Y finalmente se trocea el conjunto de entrada en entrenamiento 80% y prueba 20%

```
from sklearn.preprocessing import LabelEncoder
#le = LabelEncoder()
#y = le.fit_transform(y)
print(np.unique(y))
# Troceamos las caracteristicas de X en un 80% de entrenamiento y un 20%
de prueba
```

```
[2 3]
```

Hacer un entrenamiento con la clase GaussianNB que implementa naive-

Bayes en sk-learn

- Entrenar el modelo con X_train, y_train
- Validar con X_test, y_test utilizando la librería sklearn.metrics. Al ser una clasificación binario se pueden obtener métricas tal como exactitud, precisión y sensibilidad
- Se puede verificar que la exactitud coincide con el recuento de aciertos

Se muestra la correspondiente matriz de confusión

Repetir el entrenamiento con la clase DecissionTreeClassifier que implementa el árbol de decisión en sk-learn

- Entrenar el modelo con X_train, y_train
- Validar con X_test, y_test utilizando la librería sklearn.metrics.
- Matriz de confusión

De forma volutaria, se puede buscar la documentación de la clase RandomForestClassifier

Esta clase está implementada en la librería sklearn.ensemble

RamdomForest implementa internamente varios árboles de decisión que se ensamblan y se toma como resultado para la clasificación el voto mayoritario

Parámetros principales del constructor:

- **n_estimators** : número de árboles, 100 por defecto
- **criterion** : criterio de partición ('gini', 'entropy'), por defecto es 'gini'.

Para más información:

https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

De forma voluntaria también se puede aplicar la implementación del Bagging

Se realizarán 100 repeticiones utilizando árboles de decisión

Ahora se hace uso de N clasificadores entrenados para predecir con voto mayoritario el conjunto de test

Se usa la moda como forma de obtener la etiqueta más votada

Previous
02.4 Métodos de Clasificación - Árboles de Decisión

Next
Capítulo 3: Clustering

By Departamento de Matemática Aplicada
© Copyright 2021.